**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.



RHS COMPUTING / OCR H446 / 1.4.3 LESSON 027 · DRAFT

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

# LESSON 027 · Boolean algebra rules

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

## Do Now · Recall previous learning

Q1. Normalise $0.0110 \times 2^4$ using a two’s-complement mantissa. Give the new mantissa and exponent.

Q2. A sorted array is [3, 8, 12, 19, 27, 35, 42] . When searching for 27 , which indexes are checked, in order?

Q3. Show the array after one full left-to-right pass of ascending bubble sort on [7, 3, 5, 2] .

Q4. An alarm sounds if the door is open AND the system is armed, OR if panic is pressed. Write Q using A, B and C.

# I Do · Read new knowledge

## Key vocabulary

Recall the term — De Morgan

Recall the term — distribution

Recall the term — association

Recall the term — commutation

Recall the term — double negation

Recall the term — simplify

Recall the term — equivalent

## New learning

De Morgan — CODE:
¬(A ∧ B) → ¬A ∨ ¬B
¬(A ∨ B) → ¬A ∧ ¬B — The operator inside the brackets swaps as each term is negated.

Double negation — CODE:
¬(¬B) → B — Two consecutive NOT operations cancel.

Distributive law — CODE:
(A ∧ B) ∨ (A ∧ C)
= A ∧ (B ∨ C) — A appears in both groups. Write it once outside the brackets, then join B and C with OR.

Association + commutation — CODE:
A ∨ (B ∨ C)
= (A ∨ B) ∨ C

A ∧ B
= B ∧ A — These allow regrouping and reordering before other simplification.

## Worked example

Simplify ¬(A ∧ B)

1. Outer NOT over AND — CODE:
Use De Morgan's Law.

2. Negate each term — CODE:
A → ¬A
B → ¬B







Read it back: the voucher must be shown AND either a soft drink OR water must be purchased.

Q3. Step 1 — under-18 route: under 18 AND student card.
CODE:
A ∧ B
Step 2 — over-60 route: over 60 AND ID.
CODE:
C ∧ D
Step 3 — either route qualifies, so combine with OR.
CODE:
Q = (A ∧ B) ∨ (C ∧ D)

Q4. Step 1 — first OR gate:
CODE:
X = A ∨ B
Step 2 — second OR gate:
CODE:
Y = C ∨ D
Step 3 — final OR gate joins X and Y:
CODE:
Q = X ∨ Y
= (A ∨ B) ∨ (C ∨ D)
Because OR is associative, this also equals A ∨ B ∨ C ∨ D.

## Exit Ticket · Model answer

True. De Morgan's laws complement each input and exchange AND with OR when negating an expression.

# Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

**1. Copy the whole original question:**

**2. Write a complete new answer in GREEN PEN. Show working if needed:**

## Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

### Page 1 onwards · Do Now

Q1. $0.1100 \times 2^3$ . Shift the mantissa one place left and decrease the exponent by one to preserve the value. A normalised positive two’s-complement mantissa begins 01 ; a negative one begins 10 .

Q2. Indexes 3 → 5 → 4 (values 19 → 35 → 27). Binary search tests the middle index of the remaining sorted range, using zero-based indexing.

Q3. [3, 5, 2, 7] . Compare adjacent values and swap when the left value is larger. The largest value, 7, moves to the end.

Q4. Q = (A ∧ B) ∨ C . A = door open, B = armed, C = panic pressed. Group door AND armed first; panic is an alternative trigger.

### You Do pages · Independent questions

Q1. (i) Double negation
CODE:
¬¬A = A
Two NOT operations cancel.
(ii) De Morgan’s law
CODE:
¬A ∧ ¬B = ¬(A ∨ B)
NOT A AND NOT B means neither A nor B is true. This is an equivalent expression, not necessarily a shorter one.
(iii) De Morgan’s law, then double negation
CODE:
¬(¬A ∧ ¬B)
= ¬¬A ∨ ¬¬B
= A ∨ B

Q2. Step 1 — identify the repeated factor E.
CODE:
(E ∧ F) ∨ (E ∧ G)
Step 2 — use the distributive law.
CODE:
= E ∧ (F ∨ G)







3. AND becomes OR — CODE:
¬A ∨ ¬B

4. Equivalent behaviour — CODE:
Both are false only when A=1 and B=1.

## We Do · Practise with guidance

De Morgan

Simplify ¬A ∨ ¬B using the relevant De Morgan form.

Complete the same expression

```
¬A ∨ ¬B
= ¬(? ? ?)
```

```
¬A ∨ ¬B
= ¬(A ∧ B)
```

Double negation

Simplify ¬(¬B).

```
¬(¬B)
= ?
```

```
¬(¬B)
= B
```

Distribution

Simplify (A ∨ B) ∧ (A ∨ C).

```
(A ∨ B) ∧ (A ∨ C)
= A ∨ (? ? ?)
```

```
(A ∨ B) ∧ (A ∨ C)
= A ∨ (B ∧ C)
```

Commutation + association

Rewrite C ∨ (A ∨ B) into (A ∨ B) ∨ C.

```
C ∨ (A ∨ B)
= ?
```

```
C ∨ (A ∨ B)
= (A ∨ B) ∨ C
```

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

## You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

### Question 1

Simplify Boolean expressions
State the simplified form of each expression:
(i) ¬¬A (ii) ¬A ∧ ¬B (iii) ¬(¬A ∧ ¬B)

### Question 2

Simplify the popcorn expression
A cinema gives free popcorn when a voucher is shown and either a soft drink or a bottle of water is purchased.
E = voucher shown; F = soft drink bought; G = water bought.
Simplify (E ∧ F) ∨ (E ∧ G) .

### Question 3

Cinema discount: Boolean expression
A discounted cinema ticket is issued if the customer is under 18 AND has a student card , OR is over 60 AND has proof of age .
A = under 18; B = student card; C = over 60; D = ID. Write the Boolean expression for the output Q (discount ticket).

### Question 4

From a gate circuit to an expression
Two OR gates take inputs A, B and C, D respectively. Their outputs feed a third OR gate. Write the complete Boolean expression for the circuit output.





**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

## Exit Ticket · Answer before checking

De Morgan's Laws swap AND and OR while negating each term.